PHP et développement web dynamique

Informatique — Licence 2 (L2)

Cours INF-L2-03

Institut Africain de Management (IAM) — Bamako, Mali
Bibliothèque Universitaire — Bronbah Tech

À propos de ce cours
Passer des pages statiques aux sites dynamiques avec PHP : syntaxe du langage, formulaires GET et POST, sessions et cookies, connexion à MySQL avec PDO, requêtes préparées et construction complète d'un mini-site avec espace membre.

Document pédagogique original créé pour les étudiants de l'IAM : leçons détaillées, exemples commentés, travaux dirigés et pratiques, exercices corrigés, QCM d'évaluation et vidéos recommandées. Aucun contenu n'est reproduit d'ouvrages existants.

Sommaire

• Introduction à la matière
• Objectifs d'apprentissage
• Leçon 1 — Syntaxe PHP : variables, tableaux et structures de contrôle
• Leçon 2 — Formulaires HTML et données GET et POST
• Leçon 3 — Sessions, cookies et authentification simple
• Leçon 4 — PHP et MySQL : connexion avec PDO
• Leçon 5 — Requêtes préparées et sécurité des formulaires
• Leçon 6 — Architecture d'un mini-site dynamique
• Leçon 7 — Projet : site complet avec espace membre
• Travaux dirigés (TD)
• Travaux pratiques (TP)
• Exercices corrigés supplémentaires
• QCM d'évaluation
• Résumé — les notions importantes
• Vidéos associées aux leçons
• Sources et références

Comment utiliser ce cours : lis chaque leçon dans l'ordre, refais les exemples à la main, tente chaque exercice AVANT de lire son corrigé, puis termine par le QCM pour vérifier ce que tu as retenu.

Introduction à la matière

En première année, tu as appris à construire des pages HTML et à les décorer avec CSS : le résultat était beau, mais figé. Une page statique affiche exactement le même contenu à tous les visiteurs, et pour la modifier, il faut réécrire le fichier à la main. Le web réel fonctionne autrement : les actualités d'un site d'information, la liste des livres d'une bibliothèque ou le solde d'un compte sont produits à la demande, au moment où le visiteur arrive, à partir de données rangées dans une base. C'est ce passage du statique au dynamique que réalise PHP, le langage côté serveur le plus répandu au monde.

PHP s'exécute sur le serveur, jamais dans le navigateur : le visiteur demande une page, le serveur lance le script, le script interroge éventuellement une base de données, fabrique du HTML sur mesure, puis envoie le résultat. Le navigateur ne voit que le HTML final, pas le code source du programme. Cette architecture client-serveur, que tu as étudiée en réseaux, devient ici concrète : chaque leçon ajoute une brique — recevoir des données avec les formulaires, mémoriser un visiteur avec les sessions, stocker durablement avec MySQL, puis assembler le tout dans un site complet.

Ce cours accorde une place centrale à la sécurité, dès le premier exemple. Un site dynamique reçoit des données d'inconnus : un champ de formulaire peut contenir une tentative d'injection SQL, un mot de passe ne doit jamais dormir en clair dans la base, un texte saisi par un visiteur ne doit pas devenir du code dans la page d'un autre. Tu apprendras donc les bons réflexes en même temps que les mécanismes : requêtes préparées PDO, password_hash et password_verify, htmlspecialchars à l'affichage. À la

fin du cours, le projet final réunira toutes ces compétences dans un site avec inscription, connexion et espace membre protégé.

Objectifs d'apprentissage

À la fin de ce cours, tu seras capable de :

- Écrire des scripts PHP utilisant variables, tableaux indexés et associatifs, conditions, boucles et fonctions.
- Récupérer, contrôler et valider les données d'un formulaire transmises en GET ou en POST.
- Créer un espace membre simple avec les sessions, les cookies et une authentification protégée par password_hash.
- Se connecter à une base MySQL avec PDO et régler correctement ses options (exceptions, mode de récupération).
- Protéger les requêtes SQL contre les injections grâce aux requêtes préparées, systématiquement.
- Neutraliser les failles XSS en échappant chaque affichage de donnée externe avec htmlspecialchars.
- Organiser un mini-site dynamique en fichiers séparés (configuration, traitements, pages) et le faire fonctionner de bout en bout.

Leçon 1 — Syntaxe PHP : variables, tableaux et structures de contrôle

Un fichier PHP est un fichier texte, d'extension .php, qui mélange du HTML ordinaire et des blocs de code délimités par <?php et ?>. Tout ce qui se trouve hors des balises est envoyé tel quel au navigateur ; tout ce qui est dedans est exécuté par le serveur. La commande echo affiche du texte ou le contenu d'une variable dans la page produite. Chaque instruction se termine par un point-virgule, et les blocs sont entourés d'accolades, comme en C ou en Java : la syntaxe te semblera familière, mais PHP est plus permissif sur les types, ce qui le rend rapide à écrire… et parfois piégeux.

Les variables commencent toujours par le symbole dollar : $nom, $age, $prix. On ne déclare pas leur type à l'avance : PHP le déduit de la valeur rangée (entier, décimal, chaîne de caractères, booléen). Les chaînes peuvent s'écrire entre guillemets simples, où tout est littéral, ou doubles, où les variables à l'intérieur sont remplacées par leur valeur — c'est l'interpolation. Pour assembler des chaînes et des variables, on utilise l'opérateur point : echo « Bonjour » . $nom ;. Les comparaisons demandent de la vigilance : == compare les valeurs après conversion éventuelle, tandis que === exige la même valeur ET le même type ; préfère === dès qu'un doute existe, notamment avec des données venues d'un formulaire.

Les tableaux PHP sont la structure vedette du langage : un seul type de tableau sert à la fois de liste numérotée (indices 0, 1, 2…) et de dictionnaire à clés nommées — le tableau associatif, par exemple [« nom » => « Awa », « filiere » => « Informatique »]. On les parcourt élégamment avec la boucle foreach, qui visite chaque élément en donnant la clé et la valeur, sans gérer d'indice à la main. Les structures de contrôle

reprennent ce que tu connais : if, elseif, else, switch pour les choix ; for, while et foreach pour les répétitions. Enfin, les fonctions se déclarent avec le mot-clé function, acceptent des paramètres et peuvent retourner une valeur : elles seront indispensables dès la leçon 6 pour éviter de dupliquer le code entre les pages.

L'essentiel
PHP s'exécute sur le serveur entre les balises <?php et ?> et produit du HTML. Variables préfixées par $, tableaux à la fois listes et dictionnaires parcourus par foreach, et comparaison stricte === à privilégier.

Exemple — Un bulletin de notes généré en PHP

```
<?php
// Un bulletin de notes simple en PHP
$etudiant = "Awa Diallo";
$notes = [
"Algorithmique" => 14.5,
"PHP" => 12.0,
"Base de donnees" => 16.0,
];

function moyenne(array $notes): float {
$somme = 0;
foreach ($notes as $note) {
$somme += $note;
}
return $somme / count($notes);
}

echo "<h1>Bulletin de " . htmlspecialchars($etudiant) . "</h1>";
echo "<ul>";
foreach ($notes as $matiere => $note) {
echo "<li>" . htmlspecialchars($matiere) . " : " . $note . "/20</li>";
}
echo "</ul>";

$moy = moyenne($notes);
echo "<p>Moyenne generale : " . number_format($moy, 2) . "/20</p>";

if ($moy >= 10) {
echo "<p>Decision : admis(e)</p>";
} else {
echo "<p>Decision : ajourne(e)</p>";
}
?>
```

À retenir

• Tout code PHP vit entre <?php et ?> ; le reste du fichier est du HTML envoyé tel quel.

• Une variable commence par $, son type est déduit de sa valeur, et === compare valeur et type.

• foreach ($tableau as $cle => $valeur) est la façon naturelle de parcourir un tableau associatif.

Exercice

Écris un script PHP qui déclare un tableau associatif représentant un livre (titre, auteur, prix en FCFA, stock), affiche ces informations dans une petite liste HTML, puis indique « Disponible » si le stock est supérieur à 0 et « Rupture de stock » sinon. Ajoute une fonction prixTotal($prix, $quantite) qui retourne le montant d'une commande, et affiche le total pour 3 exemplaires.

✔ Corrigé

Le tableau se déclare ainsi : $livre = ["titre" => "Algorithmique pas a pas", "auteur" => "M. Traore", "prix" => 3500, "stock" => 7];. L'affichage parcourt le tableau : foreach ($livre as $cle => $valeur) { echo "
" . htmlspecialchars($cle) . " : " . htmlspecialchars((string)$valeur) . "
"; } — note le htmlspecialchars systématique, même sur nos propres données, pour prendre le réflexe. Le test de disponibilité est un simple if ($livre["stock"] > 0). La fonction : function prixTotal($prix, $quantite) { return $prix * $quantite; } puis echo prixTotal($livre["prix"], 3); affiche 10500. Point de vigilance : les clés du tableau associatif s'écrivent entre guillemets et la flèche => sépare la clé de sa valeur ; oublier le $ devant $livre est l'erreur la plus fréquente des débutants.

Leçon 2 — Formulaires HTML et données GET et POST

Un formulaire HTML devient intéressant quand PHP récupère ce que le visiteur a saisi. Le formulaire déclare deux choses essentielles : l'attribut action, qui indique quel script traitera les données (le fichier lui-même si on laisse action vide), et l'attribut method, qui choisit comment les données voyagent. Chaque champ possède un attribut name : c'est sous ce nom que PHP retrouvera la valeur. Quand le visiteur clique sur le bouton d'envoi, le navigateur construit une requête vers le script, et PHP range automatiquement les valeurs reçues dans des tableaux superglobaux, disponibles dans tout le script sans déclaration.

La méthode GET colle les données à la fin de l'adresse, sous la forme page.php?nom=Awa&ville=Bamako : elles sont visibles par tous, limitées en taille, et restent dans l'historique. Elle convient aux recherches et aux liens partageables, jamais aux mots de passe ni aux actions qui modifient des données. La méthode POST envoie les données dans le corps de la requête, invisibles dans l'adresse et sans limite pratique de taille : c'est le choix obligatoire pour les inscriptions, les connexions et tout envoi sensible. Côté PHP, on lit $_GET["nom"] ou $_POST["nom"] — mais attention, ces tableaux ne contiennent que ce que le visiteur a bien voulu envoyer : un champ absent provoque une erreur si on le lit sans précaution.

La règle d'or du traitement de formulaire : ne jamais faire confiance aux données reçues. On commence par vérifier que le formulaire a été soumis — par exemple en testant que $_SERVER["REQUEST_METHOD"] vaut « POST » — puis que chaque champ existe avec isset() et n'est pas vide. On nettoie avec trim() pour enlever les espaces

parasites, et on valide le format : filter_var($email, FILTER_VALIDATE_EMAIL) pour un courriel, un test de longueur ou de plage pour les nombres. En cas d'erreur, on réaffiche le formulaire avec un message clair et les valeurs déjà saisies, échappées par htmlspecialchars, pour que le visiteur n'ait pas tout à retaper. Ce cycle recevoir → contrôler → traiter ou signaler est le cœur de tous les scripts des leçons suivantes.

L'essentiel
Le navigateur envoie les champs du formulaire, PHP les reçoit dans $_GET (données dans l'URL, pour les recherches) ou $_POST (données dans le corps, pour les actions et les secrets). Toujours vérifier l'existence et la validité des champs avant de les utiliser.

Exemple — Inscription à un atelier : formulaire et traitement dans la même page

```
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>Inscription atelier</title></head>
<body>
<h1>Inscription a l'atelier web</h1>
<?php
$erreurs = [];
$nom = "";
$email = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
$nom = trim($_POST["nom"] ?? "");
$email = trim($_POST["email"] ?? "");
if ($nom === "") {
$erreurs[] = "Le nom est obligatoire.";
}
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
$erreurs[] = "Adresse e-mail invalide.";
}
if (empty($erreurs)) {
echo "<p>Merci " . htmlspecialchars($nom)
. ", inscription recue !</p>";
}
}
foreach ($erreurs as $e) {
echo "<p style='color:red'>" . htmlspecialchars($e) . "</p>";
}
?>
<form method="post" action="">
<label>Nom :
<input type="text" name="nom"
value="<?php echo htmlspecialchars($nom); ?>">
</label><br>
<label>E-mail :
<input type="email" name="email"
value="<?php echo htmlspecialchars($email); ?>">
</label><br>
<button type="submit">S'inscrire</button>
</form>
</body>
</html>
```

À retenir

• GET = données dans l'URL (recherche, liens) ; POST = données dans le corps (inscriptions, mots de passe).

• isset() et l'opérateur ?? évitent les erreurs quand un champ est absent.

• La validation côté serveur est obligatoire : le HTML (required, type=email) aide l'utilisateur mais ne protège pas le site.

Exercice

Crée un formulaire de recherche de livres avec un seul champ « motcle » envoyé en GET. Le script affiche « Vous cherchez : … » suivi du mot-clé saisi, correctement échappé. Explique ensuite pourquoi la méthode GET est ici le bon choix, et ce qui se passerait si un petit plaisantin saisissait comme mot-clé le texte <script>alert('piège')</script> sans protection.

✔ Corrigé
Le formulaire : <form method="get" action=""> <input type="text" name="motcle"> <button>Rechercher</button> </form>. Le traitement : $motcle = trim($_GET["motcle"] ?? ""); puis echo " Vous cherchez : " . htmlspecialchars($motcle) . " ";. GET est adapté parce qu'une recherche ne modifie rien sur le serveur et que son adresse peut être copiée, partagée ou mise en favori : recherche.php?motcle=php est un lien complet et honnête. Sans htmlspecialchars, le texte saisi serait recopié tel quel dans la page : le navigateur de la victime l'interpréterait comme du vrai code JavaScript et l'exécuterait — c'est la faille XSS. Avec htmlspecialchars, les caractères < et > deviennent des entités inoffensives et le piège s'affiche comme du texte ordinaire. Retiens la règle : toute donnée venue de l'extérieur (formulaire, URL, base de données) est échappée au moment de l'affichage.

Leçon 3 — Sessions, cookies et authentification simple

HTTP est un protocole sans mémoire : chaque page demandée est traitée comme si le visiteur arrivait pour la première fois. Pourtant, un site doit reconnaître un membre connecté d'une page à l'autre. Les cookies apportent un premier élément de réponse : avec setcookie(), le serveur demande au navigateur de conserver un petit texte — par exemple une préférence de langue — et de le renvoyer à chaque requête suivante. On lit ces valeurs dans $_COOKIE. Mais un cookie vit chez le visiteur : il peut le lire, le modifier ou le supprimer, et sa taille est minuscule. On n'y stockera donc jamais un mot de passe ni une information dont la falsification serait grave.

La session résout le problème proprement : les données restent sur le serveur, et seul un identifiant aléatoire et imprévisible voyage dans un cookie chez le visiteur. On démarre le mécanisme par session_start(), qui doit être appelé avant le moindre affichage — avant tout HTML, tout echo, même un simple espace en tête de fichier, sinon PHP protestera que les en-têtes sont déjà envoyés. Ensuite, on range et on relit

des valeurs dans le tableau $_SESSION comme dans n'importe quel tableau : $_SESSION["login"] = "awa";. À chaque page, session_start() retrouve automatiquement les données du visiteur grâce à son identifiant. Pour fermer proprement une session lors d'une déconnexion, on vide le tableau, on expire le cookie de session et on appelle session_destroy().

L'authentification assemble ces pièces : un formulaire de connexion en POST envoie un identifiant et un mot de passe ; le script cherche le membre correspondant et compare le mot de passe saisi à l'empreinte stockée grâce à password_verify() — la leçon 5 expliquera pourquoi on ne stocke jamais le mot de passe lui-même. Si la vérification réussit, on régénère l'identifiant de session avec session_regenerate_id(true) pour empêcher un attaquant d'imposer son propre identifiant, puis on note le login dans $_SESSION et on redirige vers l'espace membre avec header("Location: ...") suivi de exit. Chaque page protégée commence alors par le même garde du corps : session démarrée, test de isset($_SESSION["login"]), et redirection vers la connexion si le test échoue. C'est ce trio connexion → garde → déconnexion qui structure tous les espaces membres, y compris celui du projet final.

L'essentiel
Le cookie vit chez le visiteur (préférences anodines), la session vit sur le serveur et n'échange qu'un identifiant. session_start() avant tout affichage, identifiant régénéré après connexion, et garde isset($_SESSION) en tête de chaque page protégée.

Exemple — Connexion avec session et page protégée

```
<?php
// ===== Fichier 1 : connexion.php =====
session_start();
$hashDemo = password_hash("iam2026", PASSWORD_DEFAULT);
$message = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
$login = trim($_POST["login"] ?? "");
$mdp = $_POST["mdp"] ?? "";
if ($login === "etudiant" && password_verify($mdp, $hashDemo)) {
session_regenerate_id(true); // nouvel identifiant de session
$_SESSION["login"] = $login;
header("Location: espace.php");
exit;
}
$message = "Identifiants incorrects.";
}
?>
<form method="post">
<input name="login" placeholder="Login">
<input type="password" name="mdp" placeholder="Mot de passe">
<button>Se connecter</button>
</form>
<p><?= htmlspecialchars($message) ?></p>

<?php
// ===== Fichier 2 : espace.php =====
session_start();
if (!isset($_SESSION["login"])) {
header("Location: connexion.php");
exit;
}
echo "Bienvenue " . htmlspecialchars($_SESSION["login"]);
?>
```

À retenir

- setcookie() doit aussi être appelé avant tout affichage, exactement comme session_start().
- Après une connexion réussie : session_regenerate_id(true), puis on remplit $_SESSION, puis redirection + exit.
- Déconnexion complète = vider $_SESSION, expirer le cookie de session, session_destroy().

Exercice

Un camarade se plaint : sa page protégée affiche « Warning: session_start(): Session cannot be started after headers have already been sent », et n'importe qui peut ouvrir sa page membre sans se connecter. Son fichier commence par une ligne vide puis du HTML, et le test de session n'existe que dans la page de connexion. Explique les deux problèmes et donne la correction complète pour sa page membre.

✔ Corrigé
Premier problème : la ligne vide et le HTML situés avant session_start() sont déjà envoyés au navigateur quand PHP veut démarrer la session ; or une session démarre en envoyant un en-tête (le cookie d'identifiant), ce qui devient impossible. Correction : le fichier doit commencer par <?php suivi immédiatement de session_start(), sans aucun caractère — ni espace, ni ligne vide, ni HTML — avant la balise ouvrante. Deuxième problème : vérifier l'identité uniquement à la connexion ne protège rien, car rien n'empêche de taper directement l'adresse de la page membre. Correction : chaque page protégée répète le garde au tout début : session_start(); if (!isset($_SESSION["login"])) { header("Location: connexion.php"); exit; }. Le exit est indispensable : sans lui, le reste de la page continuerait de s'exécuter et d'être envoyé malgré la redirection. Règle générale : la sécurité se vérifie sur chaque page sensible, jamais une seule fois à l'entrée du site.

Leçon 4 — PHP et MySQL : connexion avec PDO

Un site dynamique range ses données dans un système de gestion de bases de données : pour PHP, le compagnon historique est MySQL (ou son cousin libre MariaDB, interchangeable dans ce cours). Tu sais déjà, depuis le cours de SQL, créer des tables et écrire des requêtes SELECT, INSERT, UPDATE et DELETE. Il reste à brancher PHP sur la base. PHP propose plusieurs extensions pour cela ; la référence moderne est PDO (PHP Data Objects), une interface uniforme qui dialogue avec MySQL, PostgreSQL ou SQLite avec le même code. Maîtriser PDO, c'est pouvoir changer de base plus tard sans réécrire toutes les requêtes.

La connexion se crée en instanciant la classe PDO avec trois informations : le DSN, une chaîne qui décrit le serveur et la base visée — par exemple mysql:host=localhost;dbname=iam_biblio;charset=utf8mb4 —, puis le nom d'utilisateur et le mot de passe de la base. Le quatrième argument, un tableau d'options, est capital : PDO::ATTR_ERRMODE réglé sur PDO::ERRMODE_EXCEPTION fait échouer bruyamment toute requête fautive au lieu de la laisser passer en silence ; PDO::ATTR_DEFAULT_FETCH_MODE sur PDO::FETCH_ASSOC fait retourner les lignes sous forme de tableaux associatifs, très agréables à manipuler ; et PDO::ATTR_EMULATE_PREPARES à false garantit de vraies requêtes préparées côté serveur, ce que la leçon suivante exploitera.

Toute connexion peut échouer — serveur arrêté, mauvais mot de passe, base inexistante — et l'échec lève une exception PDOException. On enferme donc la création de l'objet dans un try/catch : en développement, on affiche le message pour déboguer ; en production, on journalise l'erreur et on montre une page neutre, car le message technique révèle des informations précieuses pour un attaquant. Le code de connexion étant identique sur toutes les pages, on l'écrit une seule fois dans un fichier, par exemple config/connexion.php, que chaque page inclut avec require. Les résultats se lisent ensuite simplement : query() pour une requête sans paramètre extérieur, fetch() pour récupérer une ligne, fetchAll() pour toutes les lignes, puis des boucles foreach pour les afficher — toujours échappées par htmlspecialchars.

L'essentiel
PDO unifie l'accès aux bases : new PDO(dsn, utilisateur, mot de passe, options). Options indispensables : exceptions activées, récupération en tableaux associatifs, préparations réelles. Connexion dans un fichier unique inclus partout, protégée par try/catch.

Exemple — Connexion PDO et affichage d'une liste d'étudiants

```
<?php
// config/connexion.php : la connexion, ecrite une seule fois
try {
$pdo = new PDO(
"mysql:host=localhost;dbname=iam_biblio;charset=utf8mb4",
"root",
"",
[
PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
PDO::ATTR_EMULATE_PREPARES => false,
]
);
} catch (PDOException $e) {
// En production : journaliser $e->getMessage(), ne pas l'afficher
die("Connexion impossible a la base de donnees.");
}

// index.php : afficher la liste des etudiants
require "config/connexion.php";

$lignes = $pdo->query(
"SELECT id, nom, filiere FROM etudiants ORDER BY nom"
)->fetchAll();

echo "<ul>";
foreach ($lignes as $etu) {
echo "<li>" . htmlspecialchars($etu["nom"])
. " (" . htmlspecialchars($etu["filiere"]) . ")</li>";
}
echo "</ul>";
?>
```

À retenir

- Le DSN décrit le pilote, le serveur, la base et l'encodage : charset=utf8mb4 pour ne pas casser les accents.
- Sans ERRMODE_EXCEPTION, une requête fausse échoue en silence et le bug devient invisible.
- Les identifiants de la base restent dans le fichier de configuration, jamais dupliqués dans les pages.

Exercice

Sur le poste d'un étudiant, tout fonctionnait hier ; ce matin, sa page affiche un écran blanc et aucune erreur. La connexion PDO est créée sans options et son try/catch est absent. 1) Explique pourquoi l'écran est blanc alors que le serveur MySQL est simplement arrêté. 2) Réécris le bloc de connexion complet et correct. 3) Pourquoi faut-il éviter d'afficher le message exact de l'exception sur un site en production ?

✔ Corrigé
1) Sans l'option ERRMODE_EXCEPTION, PDO travaille en mode silencieux : la connexion échoue sans lever d'exception exploitable, la variable de connexion n'est pas un objet valide, et l'appel suivant provoque une erreur fatale que la configuration du serveur n'affiche pas (display_errors désactivé) : d'où la page blanche, le pire symptôme à déboguer. 2) Le bloc correct : try { $pdo = new PDO("mysql:host=localhost;dbname=iam_biblio;charset=utf8mb4", "root", "", [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION, PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, PDO::ATTR_EMULATE_PREPARES => false]); } catch (PDOException $e) { die("Connexion impossible a la base de donnees."); }. Avec les exceptions, l'arrêt de MySQL produit immédiatement un message clair pendant le développement. 3) Le message d'exception contient le DSN, parfois le nom d'utilisateur et la structure interne du serveur : autant de renseignements offerts à un attaquant. En production, on enregistre ce détail dans un journal côté serveur et l'on affiche seulement un message neutre du type « service momentanément indisponible ».

Leçon 5 — Requêtes préparées et sécurité des formulaires

Voici la faille la plus célèbre du web : l'injection SQL. Elle naît quand un programme construit une requête en collant dedans, par concaténation, un texte saisi par le visiteur. Si la requête cherche un étudiant par son nom et que le visiteur tape une valeur malicieuse contenant des apostrophes et des mots-clés SQL — la fameuse entrée ' OR '1'='1 —, le texte cesse d'être une simple donnée : il modifie la logique même de la requête, qui retourne alors tous les étudiants au lieu d'un seul. Pire, d'autres saisies peuvent lire des tables entières ou tenter de les détruire. Le problème n'est pas le visiteur malveillant, c'est le programme qui mélange le code SQL et les données dans la même chaîne de caractères.

La solution est définitive et simple : la requête préparée. On envoie d'abord à MySQL le modèle de la requête, où les emplacements des données sont marqués par des points d'interrogation ou des paramètres nommés comme :nom ; puis on exécute le modèle en fournissant les valeurs à part, avec execute(). Le serveur de base de données reçoit donc le code et les données par deux canaux séparés : quoi que contienne la valeur — apostrophes, mots SQL, guillemets —, elle sera toujours traitée comme un texte à comparer ou à stocker, jamais comme un morceau de programme. Avec PDO, la discipline est absolue : dès qu'une valeur extérieure (GET, POST, cookie, session) entre dans une requête, on prépare. query() ne sert que pour les requêtes figées qui ne contiennent aucune variable.

La sécurité d'un formulaire ne s'arrête pas à SQL. Croisillon XSS : toute donnée réaffichée passe par htmlspecialchars(), comme on l'a vu dès la leçon 2, sinon un nom d'utilisateur contenant du HTML devient du code dans le navigateur des autres visiteurs. Croisillon mots de passe : à l'inscription, on ne stocke jamais le mot de passe, ni même un simple MD5 ou SHA1, rapides et donc faciles à casser par essais massifs ; on stocke le résultat de password_hash($mdp, PASSWORD_DEFAULT), une empreinte lente à calculer et salée automatiquement, et l'on vérifie les connexions avec password_verify(). Ajoute les validations de format (filter_var, longueurs, plages) et le contrôle des types — par exemple convertir un identifiant d'URL en entier avec (int) — et tu obtiens le socle de sécurité attendu pour tout site dynamique sérieux.

L'essentiel
L'injection SQL vient de la concaténation de données dans les requêtes ; la requête préparée sépare définitivement le code des données. Compléments obligatoires : htmlspecialchars à l'affichage et password_hash/password_verify pour les mots de passe.

Exemple — Recherche préparée et inscription avec mot de passe haché

```
<?php
require "config/connexion.php";

// A NE JAMAIS ECRIRE (vulnerable a l'injection SQL) :
// $sql = "SELECT * FROM etudiants WHERE nom = '" . $_GET["nom"] . "'";

// SECURISE : la donnee reste une donnee, jamais du code SQL
$nom = trim($_GET["nom"] ?? "");
$requete = $pdo->prepare(
"SELECT id, nom, filiere FROM etudiants WHERE nom = :nom"
);
$requete->execute(["nom" => $nom]);
$etudiant = $requete->fetch();

if ($etudiant) {
echo "Filiere : " . htmlspecialchars($etudiant["filiere"]);
} else {
echo "Aucun etudiant trouve pour ce nom.";
}

// Inscription : le mot de passe est transforme en empreinte
$hash = password_hash($_POST["mdp"] ?? "", PASSWORD_DEFAULT);
$insert = $pdo->prepare(
"INSERT INTO membres (login, mdp) VALUES (:login, :mdp)"
);
$insert->execute([
"login" => trim($_POST["login"] ?? ""),
"mdp" => $hash,
]);
?>
```

À retenir

• Règle sans exception : une donnée extérieure dans une requête = requête préparée, jamais de concaténation.

• MD5 et SHA1 sont interdits pour les mots de passe ; password_hash gère le sel et la lenteur nécessaires.

• (int) sur un identifiant numérique venu de l'URL supprime toute possibilité d'y glisser du SQL.

Exercice

Le code suivant, écrit par un débutant, contient trois fautes de sécurité distinctes : $sql = "SELECT * FROM membres WHERE login = '" . $_POST["login"] . "' AND mdp = '" . md5($_POST["mdp"]) . "'"; $res = $pdo->query($sql); echo "Bienvenue " . $_GET["login"];. Identifie chaque faute, explique l'attaque qu'elle permet, puis réécris le principe correct de la vérification d'identité.

✔ Corrigé

Faute 1 : la concaténation de $_POST["login"] dans la requête ouvre l'injection SQL — un login du type ' OR '1'='1' -- transforme la condition en tautologie et connecte l'attaquant sans connaître aucun mot de passe. Faute 2 : MD5 pour le mot de passe ; MD5 est une fonction de hachage rapide conçue pour la vitesse, un attaquant qui vole la base teste des milliards de candidats par seconde, et la comparaison du mot de passe se fait en plus côté SQL au lieu d'utiliser password_verify. Faute 3 : l'affichage de $_GET["login"] sans htmlspecialchars expose au XSS — un lien piégé peut injecter du JavaScript dans la page des victimes. Version correcte : on sélectionne le membre par son seul login avec une requête préparée ($req = $pdo->prepare("SELECT id, mdp FROM membres WHERE login = :login"); $req->execute(["login" => $login]); $membre = $req->fetch();), puis on compare en PHP : if ($membre && password_verify($mdpSaisi, $membre["mdp"])) { /* connexion */ }. Le mot de passe ne voyage plus jamais dans une requête et ne figure jamais dans la base autrement que sous forme d'empreinte password_hash.

Leçon 6 — Architecture d'un mini-site dynamique

Jusqu'ici, chaque script réunissait tout : connexion, traitement et affichage. Dès qu'un site dépasse trois pages, ce mélange devient ingérable : modifier le mot de passe de la base impose d'ouvrir chaque fichier, et le même bandeau HTML est copié vingt fois avec vingt occasions de le casser. L'architecture d'un mini-site repose donc sur la séparation des rôles. Un dossier config regroupe les réglages (la connexion PDO), un dossier includes les morceaux réutilisables (l'en-tête, le pied de page, les fonctions communes), et les pages proprement dites ne contiennent que ce qui les distingue. require inclut un fichier indispensable — le script s'arrête en cas d'absence — tandis que require_once garantit une inclusion unique, précieux pour la connexion.

Un schéma fréquent consiste en un point d'entrée unique, index.php, qui reçoit un paramètre de page dans l'URL — index.php?page=cours — et inclut le bon fichier entre l'en-tête et le pied communs. Ce mécanisme est élégant mais dangereux s'il est écrit naïvement : inclure directement le nom demandé permettrait à un visiteur malin de demander des fichiers internes du serveur. On protège donc le routeur par une liste

blanche : un tableau des pages autorisées, testé avec in_array(..., true) ; toute valeur inconnue retombe sur la page d'accueil. C'est la même philosophie que pour les formulaires : ce qui vient de l'URL n'est jamais utilisé tel quel, il est comparé à une liste de possibilités connues.

Concrètement, l'arborescence d'un mini-site propre tient en quelques dossiers : la racine contient index.php et les pages d'action (connexion.php, inscription.php, deconnexion.php) ; config/ contient connexion.php ; includes/ contient entete.php, pied.php et fonctions.php ; pages/ contient le contenu de chaque rubrique ; assets/ reçoit la feuille de style et les images. Chaque page suit alors un scénario lisible : démarrer la session si besoin, inclure la configuration, traiter les formulaires éventuels AVANT tout affichage (pour pouvoir rediriger), puis produire le HTML en échappant les données. Cette régularité fait gagner un temps précieux : quand chaque fichier obéit au même plan, on sait toujours où chercher un bug.

L'essentiel
Séparer pour régner : config pour la connexion, includes pour les morceaux partagés, pages pour le contenu, assets pour le style. Un point d'entrée unique avec liste blanche pour choisir la page à inclure, jamais d'inclusion directe d'une valeur d'URL.

Exemple — Point d'entrée unique avec liste blanche

```
<?php
/*
Arborescence du mini-site :
mini-site/
index.php (ce fichier)
connexion.php inscription.php deconnexion.php
config/connexion.php
includes/entete.php pied.php fonctions.php
pages/accueil.php cours.php apropos.php contact.php
assets/style.css
*/
session_start();
require "config/connexion.php";
require "includes/fonctions.php";

$page = $_GET["page"] ?? "accueil";

// Liste blanche : seules ces pages peuvent etre incluses
$pagesAutorisees = ["accueil", "cours", "apropos", "contact"];
if (!in_array($page, $pagesAutorisees, true)) {
$page = "accueil"; // toute valeur inconnue retombe sur l'accueil
}

require "includes/entete.php"; // bandeau et menu communs
require "pages/" . $page . ".php"; // contenu de la rubrique
require "includes/pied.php"; // pied de page commun
?>
```

À retenir

• require fait échouer le script si le fichier manque : c'est le bon choix pour la configuration et la connexion.

• Le paramètre ?page= n'est utilisé qu'après passage dans la liste blanche, en comparaison stricte.

• Les traitements (POST, redirections) se font avant le premier affichage, toujours dans le même ordre.

Exercice

Un mini-site contient ces fichiers mélangés à la racine : bdd.php (la connexion), menu.html, style.css, accueil.php, liste-livres.php, detail-livre.php, login.php, verif.php (qui contient des fonctions), photo1.jpg. Propose une arborescence rangée en dossiers, indique les déplacements à faire, et précise quels fichiers les pages devront inclure et avec quelle instruction.

✔ Corrigé
Arborescence proposée : à la racine, les pages accueil.php, liste-livres.php (ou index.php en point d'entrée), detail-livre.php et login.php ; config/ reçoit bdd.php renommé connexion.php ; includes/ reçoit menu.html (renommé entete.php si l'on veut y glisser du PHP, par exemple l'affichage du login du membre connecté) et verif.php renommé fonctions.php ; assets/ reçoit style.css et photo1.jpg. Inclusions : chaque page commence par require "config/connexion.php" dès qu'elle touche à la base — require, car continuer sans connexion n'a aucun sens — et par require "includes/fonctions.php" si elle utilise les fonctions communes ; l'en-tête et le pied s'incluent au moment de produire le HTML. Les chemins dans les pages rangées à la racine restent simples ; si certaines pages vivent dans un sous-dossier, on construit les chemins depuis la racine du projet pour éviter les erreurs du type « fichier introuvable » selon le dossier courant. Test final de l'architecture : changer le mot de passe de la base ne doit exiger la modification que d'un seul fichier, config/connexion.php.

Leçon 7 — Projet : site complet avec espace membre

Le projet final assemble toutes les briques du cours dans un site cohérent : le site de la bibliothèque de l'institut. Côté visiteur, le site présente l'accueil, le catalogue des livres lu depuis la base MySQL et une page de recherche. Côté membre, un visiteur peut créer un compte avec un login et un mot de passe, se connecter, accéder à un espace personnel qui le salue par son nom et consulter la liste de ses emprunts ; un lien de déconnexion ferme proprement la session. Chaque fonctionnalité correspond à une leçon : catalogue en PDO, recherche préparée, inscription avec password_hash, connexion avec password_verify et garde de session sur les pages privées.

Le travail commence par la base de données : une table membres (id, login unique, mdp pour l'empreinte, date d'inscription) et une table livres (id, titre, auteur, annee, stock) suffisent, reliées plus tard si l'on veut gérer les emprunts. Vient ensuite le squelette : l'arborescence de la leçon 6, le fichier de connexion, l'en-tête avec un menu qui change selon que l'on est connecté ou non — c'est un excellent usage de isset($_SESSION["login"]) dans l'affichage. On développe ensuite les pages dans un

ordre sûr : d'abord les pages publiques en lecture seule, puis l'inscription, puis la connexion et l'espace membre, enfin la déconnexion. À chaque étape, le site reste fonctionnel et testable.

Les tests font partie du projet, pas de son décor. On vérifie au minimum : l'inscription refuse un login déjà pris et un mot de passe trop court ; la connexion échoue avec un mauvais mot de passe sans révéler lequel des deux champs est faux ; la page membre redirige un visiteur non connecté ; une recherche contenant ' OR '1'='1 ne retourne pas tout le catalogue ; un titre de livre contenant <b> s'affiche comme du texte et non en gras. Documenter le site fait aussi partie de la livraison : la structure de la base, les comptes de démonstration et la marche à suivre pour installer le projet sur un autre poste. Un site qui fonctionne, qui résiste aux attaques de base et qui s'explique : voilà le niveau attendu en fin de Licence 2.

L'essentiel
Le projet enchaîne : schéma de base simple, squelette d'arborescence, pages publiques, inscription hachée, connexion vérifiée, espace protégé, déconnexion. Les tests de sécurité élémentaires (injection, XSS, accès direct) font partie de la définition de « terminé ».

Exemple — Inscription d'un membre : le fichier complet

```
<?php
// inscription.php : creer un compte membre
require "config/connexion.php";
session_start();

$message = "";
if ($_SERVER["REQUEST_METHOD"] === "POST") {
$login = trim($_POST["login"] ?? "");
$mdp = $_POST["mdp"] ?? "";
if (strlen($login) < 3) {
$message = "Login trop court (3 caracteres minimum).";
} elseif (strlen($mdp) < 8) {
$message = "Mot de passe trop court (8 caracteres minimum).";
} else {
$verif = $pdo->prepare(
"SELECT id FROM membres WHERE login = :login"
);
$verif->execute(["login" => $login]);
if ($verif->fetch()) {
$message = "Ce login est deja pris.";
} else {
$hash = password_hash($mdp, PASSWORD_DEFAULT);
$ins = $pdo->prepare(
"INSERT INTO membres (login, mdp) VALUES (:login, :mdp)"
);
$ins->execute(["login" => $login, "mdp" => $hash]);
$_SESSION["login"] = $login;
header("Location: espace.php");
exit;
}
}
}
?>
<h1>Creer un compte</h1>
<p><?= htmlspecialchars($message) ?></p>
<form method="post">
<input name="login" placeholder="Login">
<input type="password" name="mdp" placeholder="Mot de passe">
<button>S'inscrire</button>
</form>
```

À retenir

- Ordre de développement sûr : base, squelette, pages publiques, inscription, connexion, espace membre, déconnexion.
- Le menu du site reflète l'état de la session : « Se connecter / S'inscrire » ou « Mon espace / Se déconnecter ».
- Un compte de démonstration et la structure SQL documentés valent autant qu'une page supplémentaire.

Exercice

Rédige le cahier de tests du projet : pour chaque fonctionnalité (inscription, connexion, page membre, recherche dans le catalogue, déconnexion), décris un test qui doit réussir et un test d'attaque ou d'erreur qui doit échouer proprement. Indique pour

chacun le résultat attendu à l'écran.

✔ Corrigé
Inscription — réussite : login nouveau et mot de passe de 10 caractères → compte créé, arrivée directe sur l'espace membre avec le message de bienvenue. Échec propre : même login réinscrit → message « Ce login est deja pris », aucun doublon en base, pas d'erreur technique affichée. Connexion — réussite : identifiants corrects → espace membre. Échec : mauvais mot de passe → message unique « Identifiants incorrects », qui ne précise pas si c'est le login ou le mot de passe, pour ne pas aider un attaquant à deviner. Page membre — accès direct à espace.php sans session → redirection vers connexion.php, aucun contenu privé visible. Recherche — recherche « php » → seuls les livres concernés ; recherche ' OR '1'='1 → aucun résultat (la chaîne est traitée comme un titre de livre), jamais tout le catalogue. Déconnexion — après clic sur « Se déconnecter », retour à l'accueil et espace.php de nouveau inaccessible. Ce cahier se rejoue après chaque modification du code : c'est le filet de sécurité du projet.

Travaux dirigés (TD)

TD 1 — Formulaire de demande de stage avec validation complète

Une entreprise partenaire de l'institut propose un formulaire de demande de stage : nom complet, adresse e-mail, numéro de téléphone, filière (liste déroulante : Informatique, BBA, Droit), message de motivation. Écris le formulaire HTML et son traitement PHP dans la même page. Contraintes : envoi en POST ; tous les champs obligatoires ; adresse e-mail contrôlée par filter_var ; filière vérifiée contre la liste des valeurs autorisées ; messages d'erreur regroupés et affichés au-dessus du formulaire ; valeurs déjà saisies réaffichées (échappées) ; en cas de succès, enregistrement de la demande dans une table demandes (nom, email, telephone, filiere, message) avec une requête préparée, puis message de confirmation sans réafficher le formulaire vide en dessous.

✔ Corrigé du TD 1
Démarche : le formulaire déclare method="post" et des attributs name identiques aux clés lues dans $_POST. Le traitement ne démarre que si $_SERVER["REQUEST_METHOD"] === "POST". On récupère chaque champ avec trim($_POST["..."] ?? ""), puis on remplit un tableau $erreurs : nom et message non vides (test === "" après trim), email validé par filter_var(..., FILTER_VALIDATE_EMAIL), téléphone contrôlé par une longueur minimale et la présence de chiffres uniquement après nettoyage, filière testée par in_array($filiere, ["Informatique", "BBA", "Droit"], true) — la liste déroulante ne dispense jamais de ce contrôle, car une requête fabriquée à la main peut envoyer n'importe quelle valeur. Si $erreurs n'est pas vide, on réaffiche le formulaire : chaque valeur est réinjectée avec value="<?php echo htmlspecialchars($nom); ?>" et la liste des erreurs est affichée échappée elle aussi. Si tout est valide, on prépare : $req = $pdo->prepare("INSERT INTO demandes (nom, email, telephone, filiere, message) VALUES (:nom, :email, :tel, :filiere, :msg)"); puis $req->execute([...]) avec les valeurs nettoyées, et l'on affiche uniquement la confirmation. Pièges à éviter : se fier à l'attribut required (contournable), afficher le message de motivation sans htmlspecialchars (XSS stocké, lu plus tard par le personnel de l'entreprise), et tenter d'insérer avant la fin des validations.

TD 2 — Mini-annuaire des étudiants en PDO

On dispose d'une table etudiants (id, nom, prenom, filiere, email) déjà remplie. Réalise trois pages : 1) annuaire.php affiche tous les étudiants triés par nom, sous forme de tableau HTML, chaque nom étant un lien vers la fiche ; 2) fiche.php reçoit un identifiant dans l'URL (fiche.php?id=12) et affiche le détail de l'étudiant, ou le message « Étudiant introuvable » ; 3) ajout.php contient un formulaire d'ajout (nom, prénom, filière, e-mail) traité en POST avec contrôle de l'e-mail et insertion préparée. Précise pour chaque page les requêtes utilisées et les protections appliquées aux données extérieures.

✔ Corrigé du TD 2
Page annuaire : la requête ne dépend d'aucune donnée extérieure, donc $pdo->query("SELECT id, nom, prenom, filiere FROM etudiants ORDER BY nom, prenom")->fetchAll() suffit ; l'affichage boucle avec foreach et échappe nom, prénom et filière ; le lien est construit ainsi : fiche.php?id= suivi de (int)$etu["id"] — l'identifiant issu de la base est fiable, mais prendre l'habitude du transtypage ne coûte rien. Page fiche : $id = (int)($_GET["id"] ?? 0); cette conversion neutralise toute tentative d'injection dans l'URL (« 12 OR 1=1 » devient 12 ou 0) ; on prépare quand même par principe : $req = $pdo->prepare("SELECT * FROM etudiants WHERE id = :id"); $req->execute(["id" => $id]); $etu = $req->fetch(); si $etu vaut false, on affiche « Étudiant introuvable » plutôt qu'une page blanche. Page ajout : même schéma que le TD 1 — vérification REQUEST_METHOD, trim, filter_var pour l'e-mail, filière en liste blanche, INSERT préparé, confirmation. Protection transversale : aucun echo de donnée (base, GET ou POST) sans htmlspecialchars, et aucun message d'exception affiché à l'écran.

Travaux pratiques (TP)

TP 1 — Construire le site de la bibliothèque du département, séance par séance

Objectif : livrer en trois séances le site complet décrit dans la leçon 7. Séance 1 — mettre en place l'environnement local (serveur web, PHP, MySQL), créer la base iam_biblio et ses tables membres et livres avec quelques lignes de démonstration, écrire config/connexion.php et vérifier qu'une page de test affiche « Connexion OK » ou le message neutre prévu. Séance 2 — construire le squelette (index.php à liste blanche, entete.php, pied.php), les pages publiques accueil et catalogue alimentées par la base, et la recherche de livres par titre. Séance 3 — ajouter inscription, connexion, espace membre (liste des livres réservés affichée à titre de démonstration) et déconnexion, puis passer le cahier de tests complet de la leçon 7. Livrables : le dossier du site, le fichier SQL de création des tables, et une fiche d'installation d'une page.

✔ Attendus du TP 1

Grille d'évaluation et blocages fréquents. Séance 1 : la connexion doit être créée avec les trois options PDO du cours ; le blocage classique est l'écran blanc — vérifier que l'affichage des erreurs est actif en développement et que MySQL est démarré avant d'accuser le code. Séance 2 : le catalogue doit échapper titres et auteurs ; la recherche utilise prepare/execute avec un paramètre nommé et le joker % ajouté côté PHP autour de la valeur, jamais concaténé dans la requête elle-même — $req->execute(["motif" => "%" . $mot . "%"]) avec une clause LIKE. Séance 3 : l'inscription doit refuser les doublons de login (contrôle préalable + colonne déclarée UNIQUE en SQL, double sécurité) ; après connexion, session_regenerate_id(true) est présent ; espace.php commence par le garde isset. Points de vigilance pour la note : aucun mot de passe en clair ou en MD5 dans la base (vérifier visuellement dans phpMyAdmin que la colonne contient des empreintes commençant par $2y$), aucune concaténation de variable dans les requêtes, aucune page privée accessible sans session. La fiche d'installation doit permettre à un camarade de faire tourner le site en moins de quinze minutes : prérequis, import du SQL, réglage du fichier de configuration, compte de démonstration.

Exercices corrigés supplémentaires

Exercice 1

Le script de connexion d'un ancien étudiant contient cette ligne : $sql = "SELECT * FROM membres WHERE login = '" . $_POST["login"] . "'";. Montre, avec un exemple de saisie précis, comment un attaquant se connecte sans mot de passe, puis réécris la recherche du membre de façon sûre.

✔ Corrigé
Si l'attaquant tape dans le champ login la valeur ' OR '1'='1' -- , la requête devient SELECT * FROM membres WHERE login = '' OR '1'='1' -- ' : la condition '1'='1' est toujours vraie, et les deux tirets commentent la fin de la requête ; la base retourne donc le premier membre de la table, souvent l'administrateur. Version sûre : $req = $pdo->prepare("SELECT id, login, mdp FROM membres WHERE login = :login"); $req->execute(["login" => trim($_POST["login"] ?? "")]); $membre = $req->fetch(); — la saisie entière, guillemets compris, est comparée comme un nom de login ; aucun membre ne portant ce nom bizarre, la recherche ne retourne rien. La vérification du mot de passe se fait ensuite avec password_verify sur l'empreinte récupérée.

Exercice 2

On veut mémoriser la langue préférée d'un visiteur (français ou anglais) pendant 30 jours, pour afficher le site directement dans la bonne langue à sa prochaine visite. Écris le code qui enregistre ce choix depuis un petit formulaire, puis celui qui lit la préférence au chargement d'une page. Explique pourquoi il faut malgré tout valider la valeur lue dans le cookie.

✔ Corrigé
Enregistrement (traitement du formulaire, avant tout affichage) : $langue = $_POST["langue"] ?? "fr"; if (!in_array($langue, ["fr", "en"], true)) { $langue = "fr"; } setcookie("langue", $langue, time() + 30 * 24 * 3600, "/"); — le quatrième argument « / » rend le cookie valable pour tout le site. Lecture en tête de page : $langue = $_COOKIE["langue"] ?? "fr"; puis le même test in_array avant d'utiliser la valeur pour choisir les textes. Validation indispensable : un cookie se modifie librement dans le navigateur, son contenu est donc une donnée extérieure comme une autre ; sans liste blanche, un visiteur pourrait y placer du HTML qui serait affiché (XSS) ou un nom de fichier qui serait inclus. On ne stocke jamais dans un cookie de mot de passe, de rôle ou de prix : uniquement des préférences sans conséquence, toujours revérifiées.

Exercice 3

La page admin.php d'un site affiche la liste des membres inscrits, mais elle est accessible à n'importe qui connaissant son adresse. De plus, le bouton « Déconnexion » actuel ne fait que revenir à l'accueil. Corrige les deux problèmes : écris le garde complet en tête de admin.php et le script deconnexion.php complet.

✔ Corrigé
Garde de admin.php, tout en haut du fichier, avant le moindre caractère affiché : <?php session_start(); if (!isset($_SESSION["login"])) { header("Location: connexion.php"); exit; } — ainsi, un visiteur sans session est renvoyé avant qu'une seule ligne de la liste ne soit produite. Script deconnexion.php : <?php session_start(); $_SESSION = []; if (ini_get("session.use_cookies")) { $p = session_get_cookie_params(); setcookie(session_name(), "", time() - 42000, $p["path"], $p["domain"], $p["secure"], $p["httponly"]); } session_destroy(); header("Location: index.php"); exit; ?>. Les trois gestes sont complémentaires : vider le tableau efface les données côté application, l'expiration du cookie efface l'identifiant chez le visiteur, et session_destroy supprime les données côté serveur. Oublier l'un des trois laisse soit des données orphelines, soit un identifiant réutilisable. Après déconnexion, un retour arrière du navigateur peut réafficher une copie en cache, mais toute nouvelle demande repart de la page de connexion : c'est le test à effectuer pour valider la correction.

QCM d'évaluation

Choisis la bonne réponse pour chaque question. Les corrigés détaillés se trouvent juste après le QCM : ne les regarde qu'après avoir répondu à tout.

1. Quelle balise ouvre un bloc de code PHP ?

   A) <php>

   B) <?php

   C) <script php>

2. Quelle est la différence essentielle entre $_GET et $_POST ?

   A) POST est toujours plus rapide que GET

   B) GET efface les données après l'envoi

   C) GET transmet les données dans l'URL, POST dans le corps de la requête

3. Pour démarrer une session, on appelle…

   A) session_start(), avant le moindre affichage

   B) session_open(), après le formulaire

   C) $_SESSION_start(), n'importe où

4. Dans new PDO("mysql:host=localhost;dbname=cours", ...), la chaîne qui commence par mysql: est…

   A) Un cookie de connexion

   B) Un nom de fichier

   C) Le DSN, qui décrit le pilote, le serveur et la base visée

5. Une requête préparée protège contre les injections SQL parce que…

   A) Elle chiffre le contenu de la base

   B) Les données sont transmises séparément du code SQL et toujours traitées comme de simples valeurs

   C) Elle interdit l'instruction SELECT

6. À l'inscription, le mot de passe d'un membre doit être stocké…

A) En clair, pour pouvoir le lui renvoyer

B) Haché avec MD5, qui est rapide

C) Transformé par password_hash(), et vérifié à la connexion par password_verify()

7. Avant d'afficher dans la page une donnée saisie par un visiteur, il faut appliquer…

A) htmlspecialchars(), pour neutraliser tout HTML injecté

B) strtoupper(), pour la mettre en majuscules

C) Aucun traitement, le navigateur s'en charge

8. Pour vérifier qu'un champ de formulaire a bien été envoyé avant de le lire, on utilise…

A) Rien : PHP crée le champ automatiquement

B) isset($_POST['nom']), ou l'opérateur ?? qui fournit une valeur par défaut

C) La fonction delete()

9. Un cookie, en PHP, est…

A) Un fichier stocké uniquement sur le serveur

B) Une variable qui survit à la fermeture du navigateur sans réglage

C) Un petit texte stocké par le navigateur, renvoyé au serveur à chaque requête, créé par setcookie()

10. Pour parcourir un tableau associatif $notes en récupérant chaque matière et sa note, on écrit…

A) foreach ($notes as $matiere => $note)

B) for ($matiere = $note)

C) repeat ($notes) until end

Corrigés du QCM

1. Réponse B — Le code PHP s'écrit entre <?php et ?> ; tout ce qui est en dehors est envoyé tel quel au navigateur.

2. Réponse C — GET convient aux recherches partageables ; POST, invisible dans l'adresse, est obligatoire pour les mots de passe et les actions qui modifient des données.

3. Réponse A — session_start() envoie l'en-tête du cookie d'identifiant : il doit précéder tout HTML, tout echo et même tout espace en tête de fichier.

4. Réponse C — Le DSN (Data Source Name) indique à PDO où et comment se connecter ; il est suivi du nom d'utilisateur et du mot de passe de la base.

5. Réponse B — Code et données voyagent par deux canaux : une saisie comme ' OR '1'='1 devient un texte comparé, jamais un morceau de requête.

6. Réponse C — password_hash produit une empreinte lente et salée ; MD5, trop rapide, laisse casser les mots de passe par essais massifs si la base fuit.

7. Réponse A — htmlspecialchars transforme < et > en entités inoffensives : la donnée s'affiche comme du texte au lieu d'être exécutée (protection anti-XSS).

8. Réponse B — Lire un index absent provoque un avertissement et un comportement bancal ; isset() et ?? sont les deux gardes standard.

9. Réponse C — Le cookie vit chez le visiteur : utile pour des préférences, il ne doit jamais contenir de secret ni de donnée à laquelle on fait confiance sans contrôle.

10. Réponse A — foreach visite chaque élément en exposant la clé et la valeur, sans gérer d'indice manuellement : c'est la boucle naturelle des tableaux PHP.

Résumé — les notions importantes

• PHP s'exécute côté serveur entre <?php et ?> : variables en $, tableaux listes ou associatifs, foreach et fonctions structurent tous les scripts.

• Formulaires : GET pour les recherches (données dans l'URL), POST pour les actions et les secrets ; validation côté serveur systématique (isset, trim, filter_var, listes blanches).

• Sessions : session_start() avant tout affichage, données dans $_SESSION côté serveur, identifiant régénéré après connexion, garde isset sur chaque page protégée, destruction complète à la déconnexion.

• PDO : connexion par DSN avec exceptions activées, tableaux associatifs par défaut et préparations réelles, le tout dans un fichier de configuration unique inclus par require.

• Sécurité SQL : requêtes préparées pour toute donnée extérieure — jamais de concaténation ; la saisie ' OR '1'='1 doit rester une donnée inoffensive.

• Sécurité des affichages et des comptes : htmlspecialchars sur chaque echo de donnée externe ; mots de passe uniquement sous forme d'empreintes password_hash, vérifiées par password_verify.

• Architecture : config, includes, pages et assets séparés ; point d'entrée avec liste blanche ; traitements avant affichage — un site lisible est un site corrigeable.

Vidéos associées aux leçons

Ces vidéos, en français, complètent les leçons. Liens vérifiés au moment de la publication de ce document.

Leçon 1 — PHP et MySQL Pour Débutant - Chapitre 05 - Les variables

Chaîne : Ecole Numérique du Génie en Programmation (ENGP) · Langue : Français · Niveau : Débutant

Cette vidéo présente les variables PHP pas à pas : déclaration avec le symbole dollar, types de valeurs et premiers affichages avec echo. Elle donne une base concrète avant d'aborder les tableaux et les structures de contrôle de la leçon 1.

https://www.youtube.com/watch?v=aPJ18O_85ec

Leçon 1 — Les tableaux en PHP : indexés, associatifs et multidimensionnels (Débutant)

Chaîne : Formateur - Thibault Chazottes - Laravel & Vue js · Langue : Français · Niveau : Débutant

On y découvre les tableaux indexés, associatifs et multidimensionnels à travers des exemples manipulés pas à pas, avec count(), print_r() et var_dump(). Parfait pour consolider la partie tableaux de la leçon 1 et préparer la boucle foreach.

https://www.youtube.com/watch?v=ICSL35yZRv8

Leçon 4 — Connexion MySQL en PHP avec PDO | Requêtes Préparées & Sécurité (Tutoriel Complet)

Chaîne : SAID GAHI · Langue : Français · Niveau : Intermédiaire

Connexion à MySQL via PDO, requêtes préparées et protection contre les injections SQL sont démontrées de bout en bout, avec la gestion des erreurs en try/catch. À regarder en parallèle des leçons 4 et 5 pour voir la syntaxe en situation réelle.

https://www.youtube.com/watch?v=1PtcBQbInBo

Leçon 3 — Partie-2_Gestion complète d'un compte utilisateur (créer espace membre / Auth complet)

Chaîne : Ronasdev · Langue : Français · Niveau : Intermédiaire

Ce tutoriel construit un système de compte utilisateur complet : inscription, connexion et déconnexion, option « se souvenir de moi » avec les cookies, et mot de passe oublié. Il illustre directement la leçon 3 et prépare le projet final du cours.

https://www.youtube.com/watch?v=mG2Mmz3VSHs

Leçon 7 — TUTO PHP - Créer un Forum complet de A à Z en PHP & MySQL ! (Formation complète)

Chaîne : FrenchCoder - Développement Web · Langue : Français · Niveau : Intermédiaire

Une formation longue qui assemble toutes les briques du cours : inscription et authentification, base de données, publication, modification et suppression de contenus. Idéale comme modèle d'architecture et de découpage pour le projet de site avec espace membre de la leçon 7.

https://www.youtube.com/watch?v=nu2m9HaeVV4

Sources et références

- Programme officiel Licence Informatique, Université IAM (Institut Africain de Management), Bamako — vérifier les intitulés d'UE auprès du secrétariat pédagogique.
- Documentation officielle du langage PHP (php.net, manuel en français) : pages de référence de PDO, des sessions, de password_hash et des superglobales $_GET, $_POST et $_SESSION.
- OWASP Foundation (owasp.org) — fiches pratiques gratuites sur la prévention des injections SQL, des failles XSS et la gestion des mots de passe.
- MDN Web Docs (Mozilla, developer.mozilla.org) — rappels sur les formulaires HTML, les méthodes GET et POST et le fonctionnement des cookies HTTP.
- Les vidéos listées dans la section « Vidéos associées aux leçons » de ce document (liens vérifiés à la publication).

Fin du cours — Bronbah Tech, Bibliothèque Universitaire IAM. Bon courage pour tes révisions !